LOCAL INFERENCE · REPLACE $200/MONTH IN AI SUBSCRIPTIONS

# Local AI audit: map your subscriptions to *free* alternatives

The best argument for local AI isn't the cost savings — it's what you build when cost stops being a constraint. Work each section with your own bill, your own tasks, your own machine. By the end you'll know exactly what can move local tonight.

NAME

DATE

ANNUAL SPEND $

## 1 Audit your current AI spend

- List **every AI tool you pay for monthly** — APIs, Copilot, cloud notebooks, hosted chat. One row each in the notes column.
- For each, capture three things: the **dollar cost**, the *primary task* you use it for, and whether that task touches **sensitive or proprietary data** (Y/N).
- Total the monthly column, then ×12. That annual number is your *baseline* — the figure every later section is measured against.

TOOL · $/MO · TASK · SENSITIVE?

## 2 Install Ollama & run your first local inference

- Install from `ollama.com`, pull a model, then run the blog's job-scoring snippet against a *real* job description and a summary of your own background:

```
ollama pull llama3
import ollama
r = ollama.chat(model='llama3', messages=[{'role':'user',
```

MODEL · LATENCY · PARSEABLE?

## 3 Classify your tasks: local vs. hosted

- Take the tool list from section 1 and tag each task `L` (local candidate) or `H` (hosted). The split rule from the blog:
- **→ L:** *batch*, *async*, or *privacy-sensitive* work — anything that runs on a schedule or where data can't leave your machine.
- **→ H:** *interactive*, latency-sensitive, non-sensitive work where speed beats everything.
- Now the payoff: **what % of your spend sits in the L bucket?** That's the number you can zero out.

% OF SPEND THAT CAN MOVE LOCAL

## 4 Build a minimal overnight automation

- Design a script that runs on a schedule, sends **zero data to external APIs**, and produces a ranked output you'd read in the morning — job listings, RSS, a folder of docs, anything real to you.

```
# 2am cron · nothing leaves the machine
for item in fetch(local_source):       # tinyfish / rss / glob
    score = ollama.chat(model='llama3', # your prompt below
```

SOURCE · PROMPT · JSON KEYS

## 5 Measure the quality gap on a real task

- Pick one task you currently pay an API for. Run the *same prompt* against a local Ollama model (7B or 13B) **and** the paid API.
- Score both outputs on **accuracy** and **usefulness**, 1–10. Then write two sentences on *where the gap is real*…
- …and one sentence on whether that gap **justifies the cost** for this specific task. That sentence is your renew-or-cancel decision.

LOCAL _/10 · PAID _/10 · VERDICT